

Session 1 — Introduction aux logiciels libres

Objectifs de la séance

- Définir « logiciel libre » et « open source ».
- Énoncer et comprendre les 4 libertés fondamentales.
- Distinguer Free Software et Open Source (philosophies, définitions).
- Identifier les logiciels libres dans son environnement quotidien.

Pourquoi ce cours ?

En 40 ans, le logiciel libre a radicalement transformé la façon dont les logiciels sont **conçus, développés, testés, déployés, vendus et maintenus**. Il structure aujourd'hui des pans entiers de l'industrie : cloud, conteneurs, données, IA. Il est aussi au cœur des débats sur l'**innovation ouverte**, la **souveraineté numérique** (européenne notamment) et l'**économie de l'immatériel**.

Ce cours aborde le logiciel libre non pas sous l'angle technique mais sous ses angles **économiques, juridiques et organisationnels** : comment fonctionnent les projets, comment ils se financent, comment ils sont régis, quelles licences s'appliquent et pourquoi. Le marché associé pèse 6 milliards d'euros en France et plus de 30 milliards en Europe, soient environ 10 % du marché informatique (logiciels et services) professionnel.

Partie 1 — Qu'est-ce qu'un logiciel libre ?

Code source vs binaire

Un logiciel existe souvent^[^1] sous deux formes : le **code source**, lisible par un humain (ou un LLM), écrit dans un langage de programmation, et le **code binaire**, produit par la compilation et exécuté par la machine. Sans le code source, on ne peut ni comprendre précisément ce que fait un logiciel, ni le corriger, ni l'adapter. C'est pour cela que l'accès au code source est une condition de base du logiciel libre, sans être une condition suffisante, comme nous allons le voir.

[^1]: Une exception notable est celle des logiciels interprétés (Python, PHP, JavaScript...).

Propriétaire vs libre

Un **logiciel propriétaire** conserve son code source fermé et impose des restrictions fortes via sa licence : pas de copie, pas de modification, usage conditionné (Windows, Photoshop, Office). Un **logiciel libre** publie son code source et garantit à l'utilisateur le droit de l'utiliser, l'étudier, le copier et le modifier (Linux, Firefox, LibreOffice).

Les 4 libertés fondamentales (FSF)

Un logiciel est **libre** s'il garantit les 4 libertés définies par Richard Stallman et la Free Software Foundation :

N°	Liberté
0	Utiliser le programme, pour n'importe quel usage
1	Étudier son fonctionnement (suppose l'accès au code source)
2	Redistribuer des copies
3	Modifier le programme et redistribuer les versions modifiées

Ces 4 libertés sont **indissociables** : si une seule manque, le logiciel n'est pas libre.

Dans une conférence filmée en France en 2012, Richard Stallman déclarait (en Français, *verbatim*):

Je peux expliquer le logiciel libre en trois mots: liberté, égalité, fraternité [...]

Liberté, parce que ce sont des logiciels qui respectent la liberté de leurs utilisateurs.

Égalité, parce que dans la communauté du logiciel libre, tous les utilisateurs sont égaux, personne n'a du pouvoir sur personne.

Et fraternité, parce que nous encourageons la coopération entre les utilisateurs.

Enfin, avec un programme, il n'y a que deux possibilités: ou les utilisateurs ont le contrôle du programme, ou le programme a le contrôle des utilisateurs.

Le premier cas s'appelle le logiciel libre, parce que pour avoir en pratique le contrôle du programme, les utilisateurs ont besoin de plusieurs libertés.

Et ces libertés sont les libertés essentielles qui font la définition, le critère du logiciel libre.

Il y en a quatre. La liberté zéro est la liberté d'exécuter le programme comme tu veux.

La liberté numéro un est la liberté d'étudier le code source du programme et de le changer pour qu'il exécute comme tu veux, pour qu'il fasse ton informatique comme tu veux.

Avec ces deux libertés, l'utilisateur individuel ou l'organisation a le contrôle seul, mais le contrôle d'un seul utilisateur ne suffit pas.

Il faut aussi le contrôle collectif de n'importe quel groupe d'utilisateurs qui veulent coopérer.

Et pour le contrôle collectif, il faut aussi la liberté 2, de diffuser des copies exactes quand tu veux, et la liberté 3, de diffuser des versions modifiées quand tu veux.

Avec ces deux libertés aussi, n'importe quel groupe d'utilisateurs qui veulent coopérer peuvent avoir le contrôle de leur version.

Et donc avec le logiciel libre, nous avons le contrôle individuel et le contrôle collectif en parallèle.

« Free as in freedom, not as in free beer »

Le mot anglais *free* est ambigu : il signifie à la fois « gratuit » et « libre ». Le logiciel libre, c'est la seconde acception.

Conséquences pratiques :

- Un logiciel libre **peut être vendu** : vente de copies, de support, de services. La liberté n'interdit pas le commerce.
- Un logiciel **gratuit n'est pas forcément libre** : Chrome est gratuit mais propriétaire (Chromium, lui, est libre) ; WhatsApp est gratuit mais fermé.

Ce qui n'est pas libre — contre-exemples

Catégorie	Exemple	Pourquoi pas libre
Propriétaire classique	Microsoft Office	Code fermé, licence restrictive
Freeware	WhatsApp	Gratuit mais code fermé
Shareware	WinRAR	Code fermé + limitation temporelle
Source available	Elastic (SSPL), BUSL, Confluent	Code visible, licence non libre (restrictions d'usage)
Open core	GitLab EE, Elastic X-Pack	Noyau libre, extensions propriétaires payantes

Ces catégories reviendront tout au long du cours : elles sont au cœur des stratégies des éditeurs (session 9).

Cas ambigus à connaître

Quelques logiciels omniprésents ont un statut plus subtil que « libre » ou « propriétaire » :

- **VS Code** : le code source (dépôt microsoft/vscode) est sous licence MIT, mais les binaires distribués par Microsoft incluent de la télémétrie et des extensions propriétaires, sous une licence **non libre**. Le fork libre sans ces ajouts s'appelle **VSCodium**.
- **Chrome / Chromium** : Chromium est libre (BSD) ; Chrome ajoute des briques propriétaires (codecs, *Widevine*, synchronisation Google).
- **Android / AOSP** : le cœur (Android Open Source Project) est libre (Apache 2.0 + GPL pour le noyau), mais la plupart des appareils embarquent les **Google Mobile Services** propriétaires (Play Store, Maps, Gmail).
- **MongoDB** : libre à l'origine (AGPL), passé en **SSPL** en 2018 — licence considérée non libre par l'OSI et la FSF car elle impose des contraintes aux hébergeurs SaaS.
- **Red Hat Enterprise Linux** : le code est libre (GPL), mais l'accès aux binaires et aux mises à jour est conditionné à un contrat de support payant.

Ces cas limites illustrent pourquoi il faut toujours **lire la licence réelle**, pas se fier à la réputation ou au nom du projet.

Partie 2 — Free Software vs Open Source

Repères historiques (détaillés en session 2)

- **1983** : Richard Stallman lance le **projet GNU** (« GNU's Not Unix ») pour produire un système d'exploitation entièrement libre.
- **1985** : Fondation de la **Free Software Foundation (FSF)**, pour porter juridiquement et politiquement le mouvement.
- **1989** : Première version de la **GNU GPL**, qui formalise juridiquement les 4 libertés et invente le **copyleft** (session 4).
- **1991** : Linus Torvalds publie la première version de **Linux**, le noyau qui manquait à GNU.
- **1998** : Netscape libère le code de son navigateur. Dans la foulée, Eric Raymond, Bruce Perens et d'autres forgent le terme « **open source** » et fondent l'**Open Source Initiative (OSI)** pour le promouvoir auprès des entreprises.

Deux termes, deux philosophies

	Free Software (1983)	Open Source (1998)
Initiateur	Richard Stallman, FSF	Eric Raymond, Bruce Perens, OSI
Angle	Éthique, liberté des utilisateurs	Avantages pratiques, méthodologie
Discours	Mouvement social et politique	Pragmatique, <i>business-friendly</i>
Slogan	« Les utilisateurs méritent la liberté »	« Le code ouvert produit de meilleurs logiciels »

Citation de Stallman : « *Open source is a development methodology ; free software is a social movement.* »

Le terme *open source* a été forgé en 1998 pour rendre l'idée acceptable aux entreprises, qui trouvaient le mot *free* ambigu et Stallman trop clivant.

Les définitions formelles

- **Free Software Definition (FSF)** : les 4 libertés, centrée sur l'utilisateur final.
- **Open Source Definition (OSD, OSI)** : 10 critères, centrée sur la licence et la distribution.
Historiquement dérivée des **Debian Free Software Guidelines (DFSG)** rédigées par Bruce Perens en 1997.

En pratique, **les deux définitions sont quasi équivalentes** : dans la quasi-totalité des cas, un logiciel libre est open source, et réciproquement. Pour couvrir les deux, on écrit **FOSS** (*Free and Open Source Software*) ou **FLOSS** (*Free/Libre and Open Source Software*) — le « L » pour *libre* évite l’ambiguïté anglophone.

Les organisations qui incarnent ces définitions

- **FSF** (Free Software Foundation, 1985) — gardienne de la définition du logiciel libre et de la GPL. Très peu de salariés, posture éthique et militante.
 - **FSFE** (Free Software Foundation Europe), **FSFE France**, **APRIL**: pendants européen / français de la FSF (avec parfois des divergences d’opinions sur certains sujets).
- **OSI** (Open Source Initiative, 1998) — gardienne de l’Open Source Definition. Maintient la liste des licences officiellement « open source ».
- **Linux Foundation** (2007, par fusion d’OSDL et du Free Standards Group) — fondation professionnelle hébergeant Linux et des centaines d’autres projets (Kubernetes, Node.js...). Financée par de grandes entreprises, poids lourd du secteur.
- **Apache Software Foundation** (1999), **Eclipse Foundation** — autres fondations généralistes.
- **Python Software Foundation...** — fondations spécialisées par écosystème (gouvernance abordée en session 9).
- **CNLL** (Conseil National du Logiciel Libre) et **APELL** (European Association of Professional Open Source) — représentants professionnels de la filière en France et en Europe.
- Et beaucoup d’autres...

L’Open Source Definition (OSD) — les 10 critères

À retenir comme grille de lecture (on n’apprend pas par cœur) :

1. Libre redistribution (pas de redevance).
2. Code source disponible.
3. Œuvres dérivées autorisées.
4. Intégrité du code source (patches séparés possibles).
5. Pas de discrimination de personnes ou de groupes.
6. Pas de discrimination de domaines d’usage (y compris commercial).
7. La licence s’applique automatiquement, sans accord supplémentaire.
8. Non spécifique à un produit.
9. Non restrictive pour les autres logiciels distribués avec.
10. Neutre technologiquement.

Les critères 5, 6 et 9 sont ceux qu’on invoque le plus souvent pour recaler une licence « presque libre » (ex. clause « pas d’usage militaire » → viole le critère 6).

Quand la distinction importe-t-elle ?

- **Ça compte** pour les discussions philosophiques, le positionnement politique d’une organisation, la communication institutionnelle.
- **Ça ne compte pas** pour le choix technique d’un outil, la licence pratique d’un projet, l’acte de contribuer.

Dans ce cours, on utilise les deux termes de manière interchangeable, sauf mention contraire.

Partie 3 — L'open source aujourd'hui

L'open source est partout

Domaine	Exemples
OS	Linux, Android, FreeBSD
Serveurs web	Apache, Nginx
Bases de données	PostgreSQL, MySQL, MariaDB, SQLite
Langages	Python, Rust, Go
Outils dev	Git, Vim/Neovim, Eclipse
Navigateurs	Firefox, Chromium
Bureautique	LibreOffice, GIMP, Inkscape
Cloud	OpenStack, Docker, Kubernetes
IA/ML	TensorFlow, PyTorch, scikit-learn

Quelques chiffres

- **96 %** des bases de code contiennent du code open source (Synopsys 2023).
- **90 %** des entreprises utilisent de l'open source (Red Hat 2023).
- **100 %** des supercalculateurs du TOP500 et environ **60 %** des smartphones tournent sous Linux.
- **4 %** de parts de marché sur le desktop en France (Linux reste marginal côté poste de travail).
- **GitHub** : plus de 100 millions de développeurs et 400 millions de dépôts — attention, « GitHub » ≠ « open source » (beaucoup de dépôts sont privés, d'autres sans licence libre, et par ailleurs la plateforme elle-même n'est pas libre).

L'open source n'est plus une alternative marginale, c'est le standard de l'industrie.

Qui développe l'open source ?

- **Individus** : bénévoles, étudiants, freelances, passionnés.
- **Organisations** : entreprises (GAFAM, Red Hat, Intel, AMD, NVidia, SUSE, SAP, Siemens...), fondations (Apache, Linux Foundation, Eclipse...), laboratoires et universités, administrations publiques.

Chiffre à retenir : sur le noyau Linux, **plus de 80 %** des contributions viennent de développeurs **payés par des entreprises**. L'image du hacker bénévole isolé correspond de moins en moins à la réalité industrielle — elle reste vraie pour certains projets communautaires (Debian, une grande partie de l'écosystème Python, des bibliothèques critiques comme OpenSSL avant Heartbleed, cf. session 10).

Cette cohabitation entre **contributeurs salariés** et **bénévoles** crée des tensions récurrentes (priorités de développement, roadmap, gouvernance) qu'on reverra en sessions 9 et 10.

Pourquoi contribuer ?

Pour les individus : apprentissage et montée en compétence ; réputation, visibilité, portfolio public ; réseau professionnel ; plaisir et engagement personnel.

Pour les entreprises : réduction des coûts ; innovation partagée et mutualisation ; attraction et recrutement de talents ; influence sur les standards ; éviter le *vendor lock-in*.

Ces motivations seront développées en sessions 7, 9 et 10 (contribution, gouvernance, modèles économiques).

Points de vigilance / pièges fréquents

- **Gratuit ≠ libre** : c'est le premier réflexe à corriger.
- **GitHub ≠ open source** : un dépôt public sans fichier LICENSE n'est **pas** libre ; par défaut, le droit d'auteur interdit toute réutilisation. Voir le code ne donne aucun droit en soi.
- « **Source available** » ≠ **libre** : voir le code ne suffit pas ; il faut les 4 libertés (critères OSD). SSPL, BUSL, Elastic License sont *source available*, pas libres.
- **Open core** : la version communautaire est libre, la version entreprise ne l'est pas — ne pas confondre les deux.
- **Android** : le cœur (AOSP) est libre, mais la plupart des appareils embarquent des couches propriétaires (Google Mobile Services).
- « **Open** » **marketing** : « OpenAI », « Open Banking », « API ouverte »... le mot *open* est galvaudé. Toujours vérifier la licence du code, pas le nom du produit.

Ce qu'il faut retenir

1. Un **logiciel libre** garantit 4 libertés : **utiliser, étudier, redistribuer, modifier**.
2. *Free* veut dire **libre**, pas gratuit.
3. **Free Software** et **Open Source** désignent (à d'infimes détails près) les mêmes réalités techniques et juridiques, avec des philosophies différentes — que l'on peut néanmoins considérer comme complémentaires.
4. L'open source est **omniprésent** dans l'industrie logicielle.
5. Tout le monde peut **utiliser** et **contribuer**.

Pour aller plus loin

- Free Software Definition : <https://www.gnu.org/philosophy/free-sw.html>
- Open Source Definition : <https://opensource.org/osd>
- Debian Free Software Guidelines : https://www.debian.org/social_contract#guidelines
- SILL (Socle Interministériel des Logiciels Libres) : <https://code.gouv.fr/sill>
- GNU Manifesto (à lire pour la session 2) : <https://www.gnu.org/gnu/manifesto.html>